PAINEL DE PESSOAS: UM PROTÓTIPO DE PLATAFORMA DE PEOPLE ANALYTICS COM INTELIGÊNCIA ARTIFICIAL, TRATAMENTO DE DADOS E CÁLCULO DETERMINÍSTICO PARA USUÁRIOS DE NEGÓCIO

BRENO RYAN DE ANDRADE FERNANDES
JOÃO VITOR FERREIRA AZEVEDO PEREIRA
ORIENTADOR: CARLOS HENRIQUE LOUREIRO FEICHAS
COORIENTADOR: BRUNO BUSTAMANTE FERREIRA LEONOR

Resumo
Áreas de negócio, como a de Recursos Humanos (RH), dependem de processos manuais ou da área de Tecnologia da Informação para transformar planilhas em indicadores. Este trabalho teve como objetivo desenvolver e avaliar uma plataforma web que automatizou esse pipeline de dados para usuários de negócio, da leitura das planilhas à visualização em dashboards. A pesquisa fundamentou-se em conceitos de People Analytics, self-service business intelligence e alucinação em modelos de linguagem. Caracterizou-se como aplicada e quantitativa: a plataforma, executada no navegador, utilizou um modelo de linguagem para identificar a estrutura das planilhas e propor o dashboard, enquanto todos os valores foram calculados por código determinístico após validação. A verificação combinou 67 testes automatizados e uma suíte de avaliação com dez bases de teste. Na execução mais recente, o modelo identificou corretamente o papel das 19 abas e os 145 mapeamentos de colunas previstos, e os 92 gráficos propostos foram aprovados pelo validador. Concluiu-se que a separação entre decisão e cálculo foi viável, embora o corpus reduzido e a ausência de testes com usuários tenham limitado a generalização dos resultados.

Palavras-chave: People Analytics. Pipeline de dados. Modelos de linguagem. Visualização de dados. Self-service business intelligence.

Abstract
Business areas, such as Human Resources (HR), rely on manual processes or on the Information Technology department to turn spreadsheets into indicators. This work aimed to develop and evaluate a web platform that automated this data pipeline for business users, from reading the spreadsheets to visualization in dashboards. The research was based on concepts of People Analytics, self-service business intelligence and hallucination in language models. It was characterized as applied and quantitative: the platform, which ran in the browser, used a language model to identify the structure of the spreadsheets and to propose the dashboard, while every value was calculated by deterministic code after validation. Verification combined 67 automated tests with an evaluation suite of ten test datasets. In the most recent run, the model correctly identified the role of the 19 sheets and the 145 expected column mappings, and all 92 proposed charts passed the validator. Separating decision from calculation proved viable, although the small corpus and the absence of user tests limited the generalization of the results.

Keywords: *People Analytics. Data pipeline. Language models. Data visualization. Self-service business intelligence.*

1. INTRODUÇÃO

O crescimento do volume de dados gerados pelas organizações tornou os sistemas tradicionais de processamento insuficientes para atender parte das demandas de análise, e o processo de extração, transformação e carga (*Extract, Transform, Load*, ETL) passou a exigir

abordagens específicas para grandes volumes e fontes heterogêneas (BALA; BOUSSAID; ALIMAZIGHI, 2017). A construção de pipelines de dados, que encadeavam coleta, processamento e análise, passou a envolver decisões de arquitetura sobre onde e como cada etapa seria executada (SANYADANAM; SRIRAMA, 2026). Mesmo em soluções acadêmicas para armazéns de dados semânticos, a implementação tradicional do ETL dependia de extensa codificação manual (DEB NATH et al., 2017).

Essa dependência de conhecimento técnico afetou especialmente áreas de negócio que não contavam com equipes de análise. Na gestão de pessoas, o uso de dados para apoiar decisões, conhecido como *People Analytics* ou *HR Analytics*, ainda apresentava pouca sustentação em pesquisas revisadas por pares, conforme Marler e Boudreau (2017). Angrave et al. (2016) argumentaram que a forma como a área de Recursos Humanos (RH) lidava com a análise de dados dificilmente produziria mudanças significativas, o que poderia afastá-la ainda mais das decisões estratégicas. Em paralelo, as ferramentas de *self-service business intelligence* (SSBI) buscaram tornar os usuários de negócio menos dependentes da área de Tecnologia da Informação (TI) (IMHOFF; WHITE, 2011), mas pressupunham que o próprio usuário integrasse os dados e montasse os dashboards, expectativa que nem sempre se confirmou na prática (ARNABOLDI; ROBBIANI; CARLUCCI, 2021).

Os modelos de linguagem de grande escala (*Large Language Models*, LLMs) ampliaram as possibilidades de automação nesse cenário, pois passaram a interpretar textos como cabeçalhos de colunas escritos de formas diferentes e foram incorporados a pipelines de ETL para enriquecer dados (COLOMBO; BERNASCONI; CERI, 2025) e a ferramentas que geraram visualizações a partir de conjuntos de dados (DIBIA, 2023). Esses modelos, contudo, podiam gerar conteúdo fluente, mas sem respaldo nos dados de entrada, fenômeno conhecido como alucinação (JI et al., 2023), e, em sistemas nos quais pessoas e inteligência artificial (IA) interagiam, dados comprometidos podiam ter seus riscos amplificados ao longo dos processos de negócio e dos ciclos de controle (QU et al., 2026). Em uma ferramenta de apoio à decisão, um número incorreto produzido pela IA não seria apenas um erro de interface, mas um risco para a decisão tomada a partir dele.

Diante desse cenário, o objetivo deste trabalho foi desenvolver e avaliar uma plataforma que automatizou o pipeline de dados, da leitura de planilhas à visualização em dashboards, para usuários de negócio, isto é, profissionais sem formação em programação ou em modelagem de dados, mas com domínio de sua própria área, tendo a área de RH como domínio de aplicação. Os objetivos específicos foram: (i) identificar automaticamente a estrutura de planilhas de RH com nomes de colunas arbitrários e mapeá-la para um esquema interno

padronizado; (ii) implementar uma etapa determinística de limpeza e padronização dos dados, com registro das correções aplicadas; (iii) gerar dashboards a partir de especificações declarativas propostas pela IA e calculadas exclusivamente por código; (iv) proteger dados pessoais e salariais durante o processamento e o envio de informações ao provedor de IA; e (v) avaliar a precisão da IA na detecção da estrutura das bases e no planejamento dos dashboards.

O trabalho justificou-se por uma tripla relevância. Do ponto de vista científico, documentou uma arquitetura em que a IA decidiu o que exibir e o código calculou os valores exibidos, acompanhada de uma avaliação da precisão do modelo com procedimento reprodutível. Do ponto de vista social, propôs um caminho para que profissionais de áreas de negócio explorassem os próprios dados sem modelagem prévia nem intervenção de um analista técnico, em linha com os princípios do SSBI (IMHOFF; WHITE, 2011) e da gestão baseada em evidências (BARENDS; ROUSSEAU, 2018), com atenção à proteção de dados pessoais dos trabalhadores. Do ponto de vista pessoal, o desenvolvimento da plataforma permitiu aos autores aplicar, em um problema real, conhecimentos de engenharia de software, engenharia de dados e IA adquiridos na graduação em Engenharia da Computação, e aproximá-los profissionalmente da área de análise de dados.

Quanto ao método, a pesquisa caracterizou-se como aplicada, de abordagem quantitativa e com objetivos exploratórios e descritivos (MIGUEL, 2012), e consistiu no desenvolvimento de um sistema computacional verificado por testes automatizados e por uma suíte de avaliação com bases de dados de teste. De modo geral, os resultados mostraram que o modelo de linguagem interpretou corretamente a estrutura das dez bases de teste e que nenhum gráfico proposto precisou ser descartado, enquanto as limitações se concentraram no tamanho do corpus e na ausência de testes com usuários. O artigo foi organizado da seguinte forma: a Seção 2 apresentou a fundamentação teórica; a Seção 3, os materiais e métodos; a Seção 4, os testes realizados e a discussão dos resultados; a Seção 5, as considerações finais; e a Seção 6, as propostas de trabalhos futuros.

2. FUNDAMENTAÇÃO TEÓRICA

Esta seção apresentou os conceitos associados às palavras-chave do trabalho, *People Analytics*, pipeline de dados, modelos de linguagem, visualização de dados e *self-service business intelligence*, e encerrou-se com uma síntese que posicionou a plataforma em relação às abordagens discutidas.

2.1 People Analytics

Para Marler e Boudreau (2017), o *HR Analytics* consistia em uma prática de RH apoiada pela tecnologia da informação, na qual dados sobre processos de RH, capital humano, desempenho organizacional e referências econômicas externas eram submetidos a análises estatísticas, visuais e descritivas, a fim de demonstrar impacto no negócio e embasar decisões em dados. Na revisão baseada em evidências que conduziram, os autores encontraram poucos estudos revisados por pares sobre o tema, o que indicou que, apesar da atenção que o conceito recebia na prática profissional, sua base científica ainda estava em construção.

Angrave et al. (2016) apresentaram uma visão crítica sobre o mesmo movimento. Para os autores, a forma como a área de RH vinha abordando a análise de dados dificilmente produziria a transformação prometida, e, se a área não se envolvesse de maneira mais cuidadosa com o potencial e com os riscos do *big data*, essas tendências poderiam marginalizá-la ainda mais das decisões de nível estratégico e até prejudicar os interesses dos empregados. Essa crítica foi relevante para este trabalho porque reforçou que o valor da análise de pessoas dependia de reduzir a distância entre quem formulava a pergunta de negócio e quem conseguia extrair a resposta dos dados.

Barends e Rousseau (2018) situaram esse movimento na gestão baseada em evidências, segundo a qual decisões organizacionais deveriam combinar evidências de diferentes fontes, como a literatura científica, os dados da própria organização, a experiência profissional e a perspectiva das partes interessadas. Os dados internos, contudo, só se tornavam evidência utilizável quando estavam organizados, confiáveis e acessíveis a quem decidia, o que remeteu às etapas de preparação e de visualização discutidas nas subseções seguintes.

No contexto deste trabalho, o domínio de RH foi escolhido como caso de aplicação por reunir dados de natureza variada, como cadastro, desligamentos, folha de pagamento, recrutamento, pesquisas de saída e movimentações de carreira, frequentemente mantidos em planilhas com estruturas distintas, e por envolver dados pessoais que exigiam cuidados específicos de privacidade.

2.2 Pipeline de dados

Sanyadanam e Srirama (2026) descreveram o pipeline de dados como uma arquitetura de ponta a ponta que conduzia as informações da origem ao destino, encadeando etapas de coleta, processamento e análise. Ao organizar essas etapas como um fluxo contínuo, o pipeline fazia com que cada transformação fosse executada sempre da mesma forma, o que tendia a reduzir inconsistências entre execuções e a tornar os resultados reprodutíveis. Na arquitetura

proposta pelos autores, o processamento foi mantido próximo à origem dos dados, em dispositivos locais, e apenas resultados de alto nível foram enviados à nuvem, o que mostrou que a localização de cada etapa também fazia parte do projeto do pipeline.

Em boa parte dos pipelines analíticos, a etapa central foi o processo de ETL, mecanismo de integração de dados em armazéns de dados (*data warehouses*) no qual a extração obtinha as informações de fontes distintas, a transformação as limpava e padronizava e a carga as armazenava em um repositório central (BALA; BOUSSAID; ALIMAZIGHI, 2017). A transformação concentrava atividades como o tratamento de valores ausentes, a padronização de formatos, a eliminação de duplicidades e a adequação do esquema dos dados à análise posterior, e suas operações podiam variar de mapeamentos simples a agregações complexas baseadas na modelagem de cubos de dados (BOUAZIZ; NABLI; GARGOURI, 2025).

Como relatórios e dashboards eram calculados a partir dos dados transformados, a qualidade dessa etapa determinava a confiabilidade dos indicadores. Wang e Strong (1996) mostraram que, para quem consumia os dados, qualidade significava adequação ao uso, e não apenas exatidão: a partir das percepções de usuários, os autores organizaram as dimensões de qualidade em quatro categorias, a intrínseca, a contextual, a representacional e a de acessibilidade. Essa visão foi pertinente a uma plataforma para usuários de negócio, que precisavam não só de dados corretos, mas também compreensíveis e disponíveis no formato da análise que desejavam fazer.

Nas abordagens tradicionais, o ETL foi implementado por regras escritas manualmente para cada fonte. Deb Nath et al. (2017) mostraram que um *framework* programável para ETL semântico reduziu a quantidade de código escrito à mão em comparação com uma solução tradicional, embora a ferramenta continuasse destinada a desenvolvedores. Mais recentemente, LLMs passaram a ser incorporados a pipelines de ETL, como no enriquecimento de metadados de um grafo de conhecimento (COLOMBO; BERNASCONI; CERI, 2025).

A automação do pipeline alterou ainda o papel do usuário: em vez de executar manualmente cada etapa de preparação, ele passou a fornecer os dados e a revisar decisões pontuais, enquanto rotinas automatizadas executavam o restante. Essa automação dependia de regras bem definidas e de mecanismos de validação, pois dados comprometidos em uma etapa podiam ter seus riscos amplificados ao longo dos processos que dependiam deles (QU et al., 2026). Na plataforma deste trabalho, a leitura, a limpeza e o cálculo foram executados no navegador do usuário, apenas informações resumidas foram enviadas a um serviço externo, e o LLM foi empregado na etapa que mais dependia de interpretação, o mapeamento de colunas com nomes arbitrários para um esquema conhecido.

2.3 Modelos de linguagem

Qu et al. (2026) observaram que a IA orientada a dados ampliou a interação entre pessoas e sistemas inteligentes em diversos cenários. Entre as técnicas de IA, os LLMs destacaram-se pela capacidade de interpretar e gerar texto em linguagem natural, o que os tornou adequados a tarefas de interpretação semântica, como reconhecer que cabeçalhos escritos de formas diferentes se referiam ao mesmo dado ou converter uma pergunta em linguagem natural em uma consulta estruturada. Colombo, Bernasconi e Ceri (2025) aplicaram essa capacidade à construção de um pipeline de dados, no qual modelos ajustados adicionaram metadados e conhecimento derivado aos nós e às relações de um grafo de conhecimento da legislação italiana.

A mesma flexibilidade trouxe um risco conhecido, a alucinação, entendida por Ji et al. (2023) como a produção de texto que carece de sentido ou de fidelidade em relação ao conteúdo de origem. Os autores distinguiram a alucinação intrínseca, em que a saída contradizia a fonte, da extrínseca, em que a saída não podia ser verificada a partir dela. Em sistemas nos quais pessoas e IA interagiam, havia ainda o risco de que dados de entrada comprometidos tivessem seus efeitos amplificados ao longo dos processos de negócio e dos ciclos de controle, razão pela qual Qu et al. (2026) propuseram uma arquitetura de segurança centrada no fluxo de dados, organizada em seis perspectivas (desempenho, negócio, dados, algoritmo, aplicação e infraestrutura) e aplicada a um caso industrial de eletrólise de alumínio.

No caso específico da análise de dados, Rumiantsau et al. (2024) avaliaram estratégias para reduzir alucinações de LLMs ao responder consultas em linguagem natural sobre dados: a geração de saídas estruturadas, a imposição de regras estritas, o aprimoramento das instruções de sistema e a integração com uma camada semântica. Segundo os autores, essas estratégias reduziram as alucinações de forma mais eficaz do que o ajuste fino tradicional do modelo. Em comum, elas restringiram o que o modelo podia produzir e tornaram sua saída verificável antes de ser apresentada ao usuário.

Neste trabalho, esse princípio foi levado a um ponto extremo: a IA foi empregada apenas em tarefas de interpretação, e não de cálculo. O modelo identificou a estrutura das planilhas, propôs o plano do dashboard em uma especificação estruturada, sugeriu a unificação de valores equivalentes e redigiu textos sobre indicadores já calculados, enquanto todos os números foram produzidos por código determinístico, após a validação de cada especificação contra os dados reais, como detalhado na Seção 3.

2.4 Visualização de dados

Bala, Boussaid e Alimazighi (2017) apontaram que as ferramentas de *Business Intelligence* (BI) apoiavam a tomada de decisão por meio da análise de dados organizacionais e dependiam dos dados integrados pelos processos de ETL. Nesse contexto, Fernandes et al. (2025) indicaram que a visualização em dashboards interativos reduzia a carga cognitiva dos gestores e ampliava a consciência situacional, permitindo identificar antecipadamente tendências e padrões relevantes. Gráficos, indicadores e filtros facilitavam a interpretação das informações, mas não eliminavam o risco de leitura inadequada: um gráfico construído sobre dados inconsistentes, ou com uma agregação mal escolhida, transmitia uma conclusão errada com a mesma clareza visual de uma conclusão correta.

Uma forma de descrever gráficos sem programar sua renderização foram as gramáticas declarativas de visualização. No Vega-Lite, proposto por Satyanarayan et al. (2017), uma visualização era especificada em alto nível pela combinação de dados, codificações visuais e composição de vistas, e um compilador gerava automaticamente os componentes necessários para desenhá-la e torná-la interativa. Esse padrão separou a descrição do que deveria ser mostrado da forma como o gráfico era produzido.

A automação da escolha das visualizações foi estudada na literatura como recomendação automática de visualizações e como conversão de linguagem natural em visualização (*Text-to-Vis* ou NL2VIS). O sistema DeepEye propôs gerar automaticamente boas visualizações a partir de um conjunto de dados (LUO et al., 2018), e o nvBench foi construído como uma base de referência para a tarefa NL2VIS, sintetizada a partir de uma base de conversão de linguagem natural em SQL (LUO et al., 2021). Com os LLMs, ferramentas como o LIDA passaram a tratar a criação de visualizações como um processo em etapas, que resumia os dados, propunha objetivos de visualização, gerava e refinava o código dos gráficos e produzia infográficos, sem depender de uma única gramática de visualização (DIBIA, 2023).

Neste trabalho, a visualização foi gerada dentro da própria plataforma e seguiu o padrão declarativo: a IA propôs, para cada gráfico, uma especificação com a tabela de origem, os filtros, o agrupamento e a métrica, que foi validada e executada pelo código. Diferentemente de abordagens em que o modelo gerava o código do gráfico, como no LIDA (DIBIA, 2023), o modelo não escreveu código nem produziu valores, o que reduziu o espaço para erros de cálculo, ao custo de limitar as opções ao vocabulário do motor de cálculo.

2.5 Self-service business intelligence

Imhoff e White (2011), em relatório do *The Data Warehousing Institute* (TDWI), caracterizaram o *self-service* BI como as facilidades do ambiente de BI destinadas a dar autonomia aos usuários e a reduzir sua dependência da área de TI. Segundo os autores, essa autonomia se apoiava em quatro objetivos: facilitar o acesso aos dados de origem, ampliar o suporte aos recursos de análise, acelerar a implantação das soluções e oferecer interfaces mais simples, personalizáveis e colaborativas.

Arnaboldi, Robbiani e Carlucci (2021), em uma pesquisa-ação sobre a adoção dessas ferramentas na gestão de uma universidade, analisaram a promessa de que usuários não especialistas poderiam integrar dados e construir dashboards interativos sem a intervenção de especialistas. Os autores mostraram, porém, que a ideia de que o conhecimento técnico se tornaria dispensável era ilusória: o projeto trouxe à tona problemas técnicos e organizacionais latentes e exigiu que os usuários adquirissem novas competências e negociassem com a área de TI.

Em paralelo, o conceito de *citizen developer*, difundido pelo Gartner, passou a designar o funcionário que desenvolvia recursos de aplicação para si mesmo ou para outras pessoas, desde que utilizasse ferramentas que a área de TI ou as unidades de negócio não tivessem vetado (GARTNER, [2026]). O *Project Management Institute* associou esse movimento ao uso de plataformas *low-code* e *no-code* e apontou o aumento dos *backlogs* de TI, que gerava gargalos e limitava a inovação, como um dos fatores que o impulsionaram (PMI, [2026]).

As ferramentas de BI de mercado passaram a incorporar assistentes de IA, mas mantiveram a dependência de dados previamente preparados. A documentação do Copilot no Power BI afirmou que a preparação do modelo semântico, com nomes legíveis, descrições de tabelas e colunas e relacionamentos sem ambiguidade, era necessária para bons resultados, e advertiu que, sem essa preparação, as respostas do assistente poderiam ser menos úteis e conter imprecisões (MICROSOFT, [2026]). No Tableau Pulse, as métricas acompanhadas pela IA foram definidas a partir de uma fonte de dados publicada e estruturada para esse fim, por usuários com permissão para publicar conteúdo no site (TABLEAU, [2026]). Em ambos os casos, a etapa de modelagem que este trabalho procurou automatizar continuou sendo pré-requisito.

Esses referenciais sustentaram a motivação deste trabalho. A plataforma desenvolvida aplicou os princípios do SSBI ao domínio de RH, mas procurou enfrentar a limitação apontada por Arnaboldi, Robbiani e Carlucci (2021): em vez de esperar que o usuário integrasse e

modelasse os dados, a interpretação da estrutura da planilha e o planejamento do dashboard foram automatizados, e o usuário foi mantido como revisor dessas decisões.

2.6 Síntese da fundamentação teórica

Em síntese, o *People Analytics* dependia de dados internos organizados e acessíveis a quem decidia (MARLER; BOUDREAU, 2017; BARENDS; ROUSSEAU, 2018); o pipeline de dados e o ETL concentravam na transformação as operações que determinavam a qualidade desses dados (BALA; BOUSSAID; ALIMAZIGHI, 2017; WANG; STRONG, 1996); os LLMs permitiam automatizar etapas de interpretação, desde que suas saídas fossem restritas e verificadas (JI et al., 2023; RUMIANTSAU et al., 2024); as especificações declarativas separavam o que mostrar da forma de desenhar (SATYANARAYAN et al., 2017); e o SSBI reduzia a dependência da área de TI sem eliminar a necessidade de conhecimento técnico (IMHOFF; WHITE, 2011; ARNABOLDI; ROBBIANI; CARLUCCI, 2021). A Tabela 1, a seguir, comparou as abordagens discutidas com a proposta deste trabalho.

Tabela 1 – Comparação entre abordagens de geração de visualizações e a proposta deste trabalho

Abordagem	Exemplos	Entrada exigida	Papel da IA no cálculo
BI com assistente de IA	Copilot no Power BI; Tableau Pulse	Modelo semântico ou fonte de dados publicada, preparados previamente	O assistente opera sobre o modelo preparado pela equipe técnica
Recomendação e conversão de linguagem natural em visualização	DeepEye; nvBench	Conjunto de dados tabular e, no caso de NL2VIS, uma pergunta	O sistema gera a especificação do gráfico a partir dos dados ou da pergunta
Geração de visualizações por LLM	LIDA	Conjunto de dados	O LLM gera e refina o código dos gráficos
Este trabalho	**Plataforma desenvolvida**	**Planilhas de RH sem preparação prévia**	**O LLM propõe especificações declarativas; o código valida e calcula todos os valores**

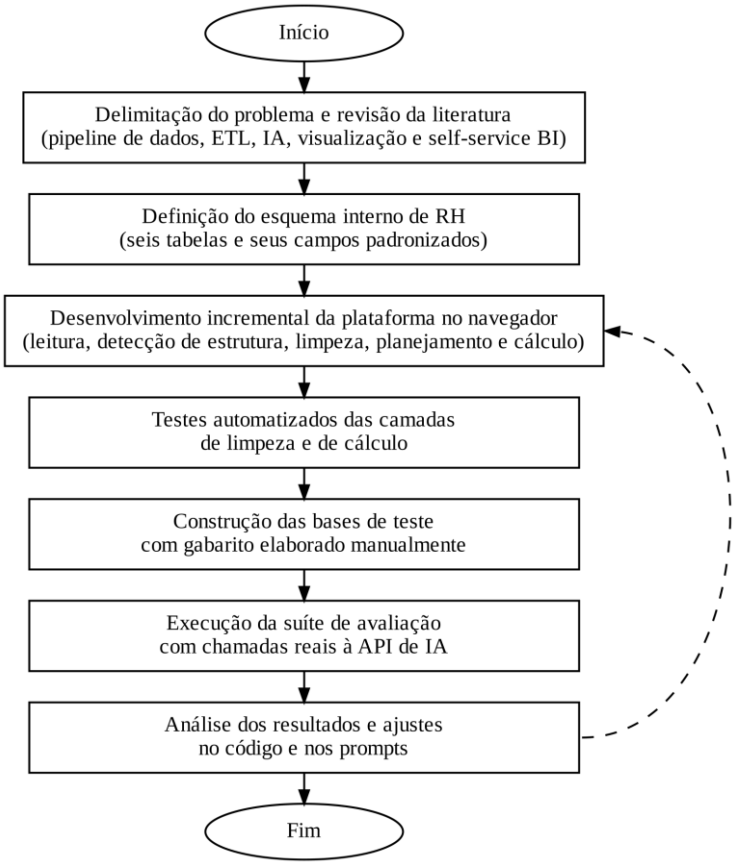
Fonte: Elaborada pelos autores (2026), com base em Microsoft ([2026]), Tableau ([2026]), Luo et al. (2018; 2021) e Dibia (2023).

Conforme a Tabela 1, as abordagens existentes exigiam dados preparados previamente ou atribuíam ao modelo a geração do código dos gráficos. A plataforma deste trabalho combinou três características: dispensou a preparação prévia dos dados por especialistas, como pretendia o SSBI, aproveitou a capacidade de interpretação dos LLMs, como nas ferramentas de geração de visualizações, e manteve o cálculo fora do alcance do modelo, o que levou ao limite a restrição das saídas proposta para reduzir alucinações (RUMIANTSAU et al., 2024).

3. MATERIAIS E MÉTODOS

Esta seção descreveu a abordagem metodológica, os requisitos, a arquitetura e as tecnologias da plataforma, a leitura das planilhas e a detecção de sua estrutura, a limpeza e a padronização dos dados, o planejamento do dashboard e o motor de cálculo, os recursos de projeção e simulação, as medidas de privacidade, a interface e os procedimentos de teste e verificação. A Figura 1, a seguir, sintetizou o percurso adotado.

Figura 1: Fluxograma geral de materiais e métodos da plataforma



Fonte: Elaborada pelos autores (2026).

Conforme a Figura 1, o trabalho partiu da delimitação do problema e da revisão da literatura, seguiu para a definição do esquema interno de RH e para o desenvolvimento incremental da plataforma e terminou com os testes automatizados, a construção das bases de teste e a execução da suíte de avaliação. A seta tracejada indicou que a análise dos resultados levou a ajustes no código e nos *prompts*, que reiniciaram o ciclo de desenvolvimento e de testes.

3.1 Abordagem metodológica

A pesquisa adotou uma abordagem aplicada, quantitativa e com objetivos exploratórios e descritivos (MIGUEL, 2012). O caráter aplicado decorreu da busca por uma solução prática para um problema real; o caráter quantitativo, do uso de métricas para avaliar a precisão da IA e o desempenho do processamento; e os objetivos exploratórios e descritivos, da investigação da viabilidade da arquitetura proposta e da descrição do comportamento do pipeline automatizado. O desenvolvimento seguiu ciclos de construção, teste e ajuste: as alterações no código ou nas instruções enviadas ao modelo foram acompanhadas da execução dos testes automatizados e, em momentos-chave, de nova execução da suíte de avaliação. A pesquisa não envolveu participantes humanos, e todas as bases utilizadas foram sintéticas ou derivadas de bases de demonstração do próprio projeto.

3.2 Requisitos da plataforma

Os requisitos foram consolidados a partir dos objetivos do trabalho, da implementação e da documentação técnica do projeto. Os requisitos funcionais compreenderam: (i) importar um ou mais arquivos XLSX, XLS ou CSV; (ii) identificar, com apoio da IA, o papel de cada aba e o mapeamento das colunas, com revisão pelo usuário; (iii) limpar e padronizar os dados e apresentar um relatório de qualidade; (iv) planejar, validar e calcular páginas, gráficos, indicadores e filtros; (v) responder a perguntas em linguagem natural com valores calculados pelo motor; (vi) oferecer projeções e simulação de cenários; (vii) permitir a edição e a exportação do dashboard; (viii) salvar projetos no navegador; e (ix) permitir a escolha do provedor de IA. Os requisitos não funcionais expressaram as restrições centrais da proposta: (i) executar inteiramente no navegador; (ii) não exibir nenhum valor numérico produzido diretamente pela IA; (iii) não enviar a base completa ao provedor e proteger dados salariais e identificadores pessoais; (iv) impedir que um item inválido interrompesse o dashboard; (v) processar planilhas com centenas de milhares de linhas; e (vi) manter as camadas de limpeza e de cálculo como funções puras, testáveis isoladamente.

3.3 Arquitetura e tecnologias

A plataforma foi desenvolvida como uma aplicação de página única (*Single Page Application*, SPA), executada inteiramente no navegador do usuário, sem servidor próprio. A interface foi construída com a biblioteca React (REACT, [2026]) e os gráficos, com a biblioteca Recharts (RECHARTS, [2026]); a leitura das planilhas utilizou a biblioteca SheetJS (SHEETJS, [2026]); e os projetos gerados foram armazenados localmente com a interface

IndexedDB do navegador (MOZILLA, [2026]). A Tabela 2, a seguir, sintetizou as tecnologias utilizadas.

Tabela 2 – Tecnologias utilizadas no desenvolvimento da plataforma

Parte	Tecnologia	Finalidade
Interface	React 18, Vite 5, Recharts e Lucide	Construção das telas, dos gráficos e dos ícones
Leitura de planilhas	SheetJS	Leitura de arquivos XLSX, XLS e CSV no navegador
Limpeza e cálculo	JavaScript (funções puras)	Tratamento dos dados, validação das especificações e cálculo dos indicadores
Inteligência artificial	APIs do Google Gemini e da Anthropic (Claude)	Detecção de estrutura, planejamento do dashboard, padronização por significado, chat e análises textuais
Armazenamento	localStorage e IndexedDB	Preferências, chave de acesso e projetos salvos no navegador
Testes	Executor de testes nativo do Node.js	Testes automatizados das camadas de limpeza e de cálculo

Fonte: Elaborada pelos autores (2026).

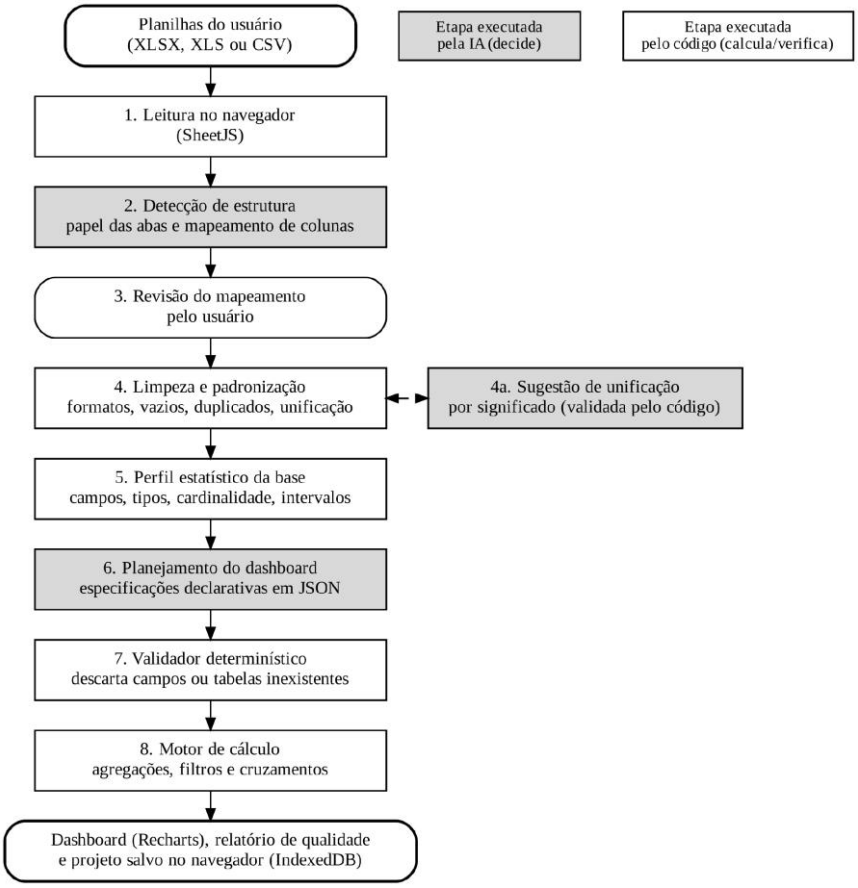
Conforme a Tabela 2, todo o processamento foi executado no navegador, e a única dependência externa durante o uso foi o provedor de IA. O código foi organizado em camadas: a interface renderizou páginas, gráficos, filtros e o chat; a orquestração coordenou o envio dos arquivos, as chamadas à IA e o acionamento do motor; a camada de IA comunicou-se com o provedor apenas nas etapas de interpretação; a camada de validação interceptou toda saída do modelo; e a camada de cálculo executou as agregações e persistiu os projetos. Nenhuma camada que gerava conteúdo por IA teve acesso direto à renderização final, pois houve sempre uma verificação determinística entre ambas.

As principais decisões de arquitetura foram registradas com suas consequências. A execução no navegador eliminou o custo de hospedagem e manteve a base no computador do usuário, mas deixou a chave de acesso à API visível nas ferramentas do navegador, o que restringiu o uso a ambientes locais. O princípio *a IA decide, o código calcula* impediu que números gerados pelo modelo chegassem ao usuário, ao custo de limitar os pedidos ao vocabulário do motor. O esquema interno fixo de seis tabelas ofereceu um vocabulário estável, mas restringiu a plataforma ao domínio de RH. Por fim, o suporte a dois provedores de IA intercambiáveis reduziu o risco de descontinuação de modelos, que havia ocorrido durante o desenvolvimento, e ofereceu uma opção gratuita. Embora a linguagem Python fosse amplamente utilizada em ferramentas de análise de dados (BAHAMÓN-MONJE; COLLAZOS-ESCOBAR; GUTIÉRREZ-GUZMÁN, 2026), sua adoção na versão principal

exigiria um processo em execução na máquina do usuário ou em um servidor; por isso, Python foi empregado apenas em uma segunda implementação do motor de cálculo, com a biblioteca Streamlit, construída para verificar de forma independente os valores calculados.

A Figura 2, a seguir, apresentou o fluxo de processamento da plataforma.

Figura 2: Fluxo de processamento da plataforma, com a separação entre as etapas executadas pela IA e pelo código



Fonte: Elaborada pelos autores (2026).

Conforme a Figura 2, no fluxo de processamento, a IA atuou em três etapas: a detecção de estrutura, a sugestão de unificação de valores e o planejamento do dashboard. Cada uma delas foi seguida de uma verificação: o mapeamento foi revisado pelo usuário, as sugestões de unificação foram validadas por regras e as especificações do dashboard passaram pelo validador antes do cálculo. As demais etapas, da leitura à geração do dashboard, foram executadas exclusivamente pelo código.

3.4 Leitura das planilhas e detecção de estrutura

O usuário podia enviar um ou mais arquivos de uma vez, nos formatos XLSX, XLS ou CSV. Após a leitura, a IA recebeu os cabeçalhos e quatro linhas de amostra de cada aba e determinou (a) se a planilha continha dados de RH, (b) qual papel cada aba desempenhava e (c) a qual campo do esquema interno cada coluna correspondia. O esquema interno foi definido como um conjunto fixo de seis tabelas, apresentado na Tabela 3, a seguir.

Tabela 3 – Esquema interno de dados de RH

Tabela	Conteúdo	Campos	Exemplos de campos
Cadastro de funcionários	Dados cadastrais e contratuais de cada pessoa	21	Matrícula, data de admissão, cargo, área, salário atual
Desligamentos	Saídas e seus motivos	7	Data, tipo e iniciativa do desligamento, tempo de casa
Folha de pagamento	Custos mensais por pessoa	8	Competência, salário base, benefícios, encargos, custo total
Recrutamento e seleção	Vagas e contratações	11	Datas de abertura e de fechamento, área, fonte de recrutamento
Entrevistas de desligamento e pesquisas	Respostas de pesquisas de saída	5	Categoria, pergunta, nota, resposta em texto
Movimentações de carreira	Mudanças de cargo, área ou salário	10	Data e tipo do evento, cargo, área e salário anteriores e novos

Fonte: Elaborada pelos autores (2026).

Conforme a Tabela 3, o esquema reuniu 62 campos, e o cadastro de funcionários, com 21 campos, serviu de base para os cruzamentos com as demais tabelas por meio da matrícula. O mapeamento proposto pela IA foi exibido ao usuário para revisão antes de qualquer processamento, e apenas campos pertencentes ao esquema foram aceitos. Colunas que não correspondiam a nenhum campo foram preservadas como métricas adicionais, abas alheias ao domínio de RH foram ignoradas, e abas com a mesma estrutura, como históricos separados por ano, receberam o mesmo papel e foram combinadas. As chamadas à IA foram repetidas automaticamente em caso de falha de rede ou de resposta malformada.

3.5 Limpeza e padronização dos dados

Confirmado o mapeamento, os dados passaram por uma etapa de limpeza executada por código determinístico. A Tabela 4, a seguir, apresentou exemplos das transformações aplicadas.

Tabela 4 – Exemplos de transformações aplicadas na limpeza e na padronização

Problema	Exemplo na planilha	Resultado
Marcador de ausência	“N/A”, “-”, “#N/D”	Valor vazio
Número com moeda, percentual ou parênteses	“R$ 1.234,56”; “12,5%”; “(1.500,00)”	1234,56; 12,5; -1500
Data em formatos variados	“05/01/2024”; “2024-01-05”	A mesma data
Competência mensal	“11/2025”; “2025/11”; “nov/25”	2025-11
Matrícula repetida	Duas linhas com a mesma matrícula	Registros combinados
Variação de grafia	“sao paulo”; “SÃO PAULO”	Forma mais bem escrita do grupo
Mesmo significado	“SP”; “São Paulo”	“São Paulo”, por sugestão da IA validada pelo código
Níveis distintos	“Analista I”; “Analista II”	Mantidos separados
Intervalo de datas	Datas de abertura e de fechamento de uma vaga	Duração em dias e faixa, como “31 a 60 dias”

Fonte: Elaborada pelos autores (2026).

Como ilustrou a Tabela 4, a maior parte das transformações dependeu apenas de regras de formato, e somente a unificação por significado envolveu a IA, que recebeu apenas os valores distintos de cada campo de categoria, com suas contagens. As sugestões foram validadas pelo código, que só uniu valores de um mesmo campo e impediu a união de níveis distintos; identificadores, nomes e respostas abertas nunca foram enviados para essa etapa. Tudo o que a limpeza alterou foi registrado em um relatório de qualidade dos dados, exibido em uma página própria, com as linhas lidas, aproveitadas e descartadas, os duplicados combinados, os valores padronizados, com o antes e o depois, e os campos com muitos valores ausentes, em linha com a visão de qualidade como adequação ao uso discutida na Seção 2.2 (WANG; STRONG, 1996).

3.6 Planejamento do dashboard e motor de cálculo

Após a limpeza, o código construiu um perfil estatístico da base, com os campos de cada tabela, seus tipos, a cardinalidade e os intervalos de valores. Esse perfil, e não os dados, foi enviado à IA, que propôs as páginas, os gráficos, os indicadores e até seis filtros globais do dashboard, cada gráfico descrito por uma especificação declarativa em JSON. A Figura 3, a seguir, apresentou um exemplo de especificação.

Figura 3: Exemplo de especificação de gráfico proposta pela IA

```
{
  "id": "g2",
  "title": "Funcionários por área",
  "chart_type": "bar_horizontal",
  "source": "funcionarios",
  "filters": [{ "field": "status", "op": "eq", "value": "Ativo" }],
  "group_by": { "kind": "category", "field": "area" },
  "metric": { "agg": "count" }
}
```

Fonte: Elaborada pelos autores (2026).

Na especificação da Figura 3, o campo *source* indicou a tabela de origem, o campo *filters* restringiu a contagem aos funcionários ativos, o campo *group_by* definiu o agrupamento por área e o campo *metric* definiu a agregação. Nenhuma especificação foi executada diretamente: um validador determinístico conferiu se cada campo e cada tabela citados existiam no perfil real da base, considerando o tipo de campo registrado no perfil e não o declarado pela IA, e descartou o item inválido sem interromper o restante do dashboard. O vocabulário do motor delimitou o que a IA podia solicitar: gráficos de barras, linha, área, pizza e radar; agregações de contagem, soma, média, mediana, mínimo e máximo; agrupamento por categoria ou por mês; filtros de igualdade, diferença, maior que, menor que e preenchimento, além de período; e o cruzamento de campos do cadastro com as demais tabelas por meio da matrícula.

Com essa separação, a IA decidiu o que mostrar, mas não produziu os números, e a mesma base e a mesma especificação resultaram sempre no mesmo valor. O mesmo princípio foi aplicado ao chat em linguagem natural, no qual a IA formulou a consulta, que foi validada e executada pelo motor, e redigiu a resposta a partir do resultado, e às análises textuais de cada gráfico, redigidas a partir de valores já calculados. Depois de gerado, o dashboard pôde ser editado pelo usuário, com a remoção de gráficos e páginas ou a solicitação de um novo gráfico em texto livre, e exportado para impressão ou PDF.

3.7 Projeções, simulador de cenários e estimativa de risco

A plataforma ofereceu ainda uma página de projeções e cenários, calculada exclusivamente pelo código. As projeções estimaram os seis meses seguintes ao último dado disponível, por regressão linear simples sobre os últimos nove meses de histórico, aplicada ao quadro de pessoal, à taxa de desligamentos e ao custo da folha. O simulador de cenários permitiu ajustar a taxa de desligamentos, o número de contratações por mês e o reajuste salarial, além de comparar o resultado com o cenário base. A estimativa de risco de desligamento

combinou, para cada categoria selecionada (como área ou cargo), as taxas históricas de desligamento observadas, com suavização para categorias com poucas pessoas.

Esses recursos foram concebidos como métodos estatísticos transparentes, e não como modelos de aprendizado de máquina treinados. A interface informou que as projeções serviam para identificar tendências, e não para prever eventos pontuais, e que a estimativa de risco descrevia padrões agregados, não devendo ser usada como critério para decisões sobre pessoas específicas. Os recursos só foram exibidos quando a base continha os dados necessários; sem registros de desligamento, por exemplo, não houve projeção de desligamentos nem estimativa de risco, e, sem custo total da folha, o custo foi calculado a partir do salário base.

3.8 Privacidade e proteção dos dados

As bases de RH continham dados pessoais e, em alguns casos, dados pessoais sensíveis, como os referentes à origem racial ou étnica e à saúde, categorias definidas pela Lei Geral de Proteção de Dados Pessoais (BRASIL, 2018). Por isso, a base completa nunca foi enviada ao provedor de IA. A plataforma realizou seis tipos de chamada à IA, e a Tabela 5, a seguir, resumiu o que foi e o que não foi transmitido em cada uma, agrupando em uma mesma linha o planejamento do dashboard e o pedido de gráfico avulso, que devolveram o mesmo tipo de resposta.

Tabela 5 – Chamadas à IA e informações transmitidas ao provedor

Chamada	Informações enviadas e resposta esperada	Informações não enviadas
Detecção de estrutura	Cabeçalhos e quatro linhas de amostra por aba, com CPFs mascarados; devolve o papel das abas e o mapeamento em JSON	Demais linhas da base
Padronização por significado	Valores distintos dos campos de categoria e suas contagens; devolve grupos equivalentes em JSON	Linhas, nomes, CPFs e matrículas
Planejamento e gráfico avulso	Perfil estatístico da base ou pedido em texto livre; devolve especificações em JSON	Exemplos de nome, CPF, matrícula e identificadores de vaga
Chat	Pergunta e resumo agregado; devolve uma consulta, executada pelo motor, e a resposta redigida a partir do resultado	Nome e salário de uma pessoa específica
Análises textuais	Valores já calculados pelo motor; devolve texto interpretativo	Registros individuais

Fonte: Elaborada pelos autores (2026).

Conforme a Tabela 5, todas as respostas da IA foram estruturadas ou se limitaram a texto redigido a partir de valores já calculados, e a exposição de dados ficou concentrada na

detecção de estrutura, cuja amostra foi composta de linhas reais, de modo que nomes de pessoas podiam ser transmitidos nessa etapa. Além disso, métricas de salário e de custo foram omitidas em grupos com menos de cinco pessoas, para que um agrupamento como “salário médio por cargo” não revelasse a remuneração de um indivíduo, e nenhum gráfico pôde agrupar ou filtrar por nome, matrícula ou CPF. A chave de acesso à API ficou armazenada apenas no navegador do usuário.

3.9 Interface da plataforma

A interface ofereceu um tutorial de cinco passos no primeiro acesso, cinco paletas de cores e os modos claro e escuro. As telas apresentadas nesta seção foram registradas com uma base sintética de teste, e a Figura 4, a seguir, apresentou a tela inicial.

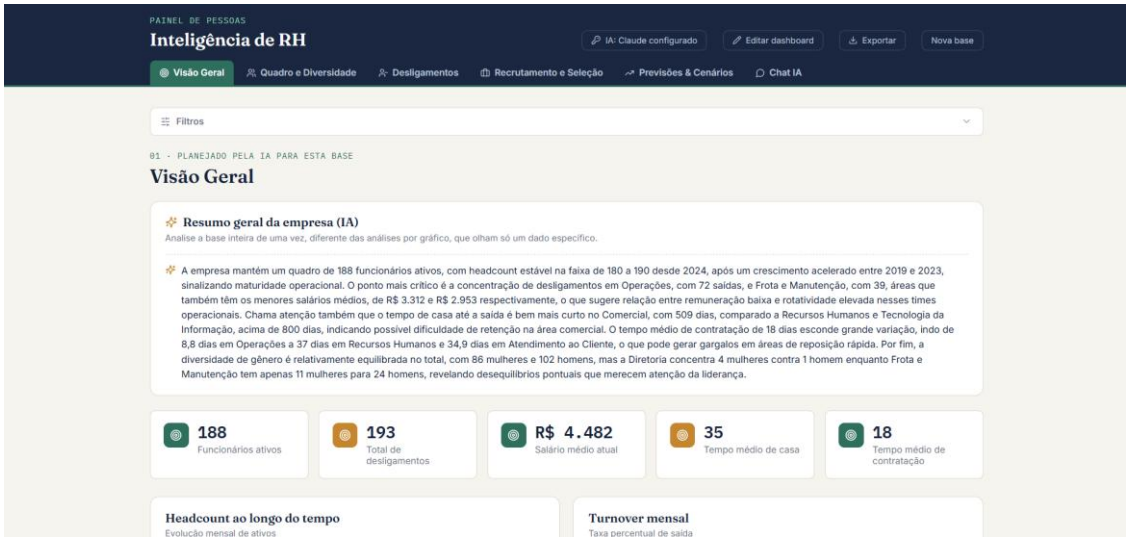
Figura 4: Tela inicial de envio de planilhas, com os projetos salvos no navegador



Fonte: Elaborada pelos autores (2026).

Na tela da Figura 4, o usuário enviou os arquivos e visualizou um aviso de que apenas uma pequena amostra seria enviada à IA, composta, como descrito na Seção 3.4, dos cabeçalhos e de quatro linhas de cada aba. A mesma tela listou os projetos salvos no navegador, que puderam ser reabertos sem repetir o envio e o processamento. A Figura 5, a seguir, apresentou a página de visão geral de um dashboard gerado.

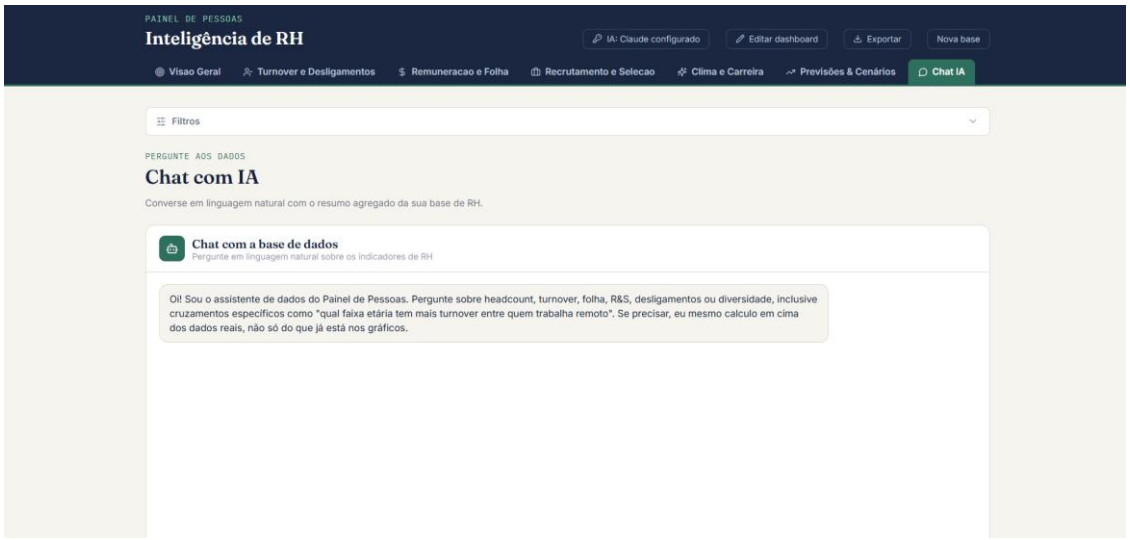
Figura 5: Página de visão geral de um dashboard gerado a partir de uma base sintética



Fonte: Elaborada pelos autores (2026).

A página da Figura 5 reuniu o resumo redigido pela IA a partir dos indicadores já calculados, os indicadores principais e os primeiros gráficos planejados para a base, e as abas no topo corresponderam às páginas temáticas propostas pela IA, entre elas a de projeções e cenários. A Figura 6, a seguir, apresentou a página de chat.

Figura 6: Página de chat com a base de dados



Fonte: Elaborada pelos autores (2026).

Na página da Figura 6, o assistente informou, já na mensagem inicial, que podia calcular cruzamentos que não apareciam nos gráficos, sempre a partir dos dados reais, conforme o fluxo descrito na Seção 3.6, em que a IA formulou a consulta e o motor de cálculo a executou.

3.10 Procedimentos de teste e verificação

A verificação combinou quatro procedimentos. O primeiro foi uma bateria de testes automatizados das camadas de limpeza e de cálculo, executada com o executor de testes nativo do Node.js (OPENJS FOUNDATION, [2026]) e com respostas da IA simuladas. O segundo foi a comparação dos valores calculados pela implementação principal, em JavaScript, com os da segunda implementação, em Python, para os mesmos dados de entrada. O terceiro foi o uso de uma base sintética de demonstração com 359 erros injetados intencionalmente, como categorias escritas de formas diferentes, linhas duplicadas, matrículas sem cadastro correspondente e reajustes que reduziam o salário, acompanhada de um gabarito que listava cada erro e sua localização. O quarto foi uma suíte de avaliação da IA com chamadas reais à API, complementada por medições de desempenho com uma base de folha de pagamento replicada em tamanhos maiores.

A suíte de avaliação utilizou dez bases de teste construídas manualmente, cada uma com um gabarito que definia se a base era de RH, o papel esperado de cada aba e o mapeamento esperado das colunas. A Tabela 6, a seguir, apresentou essas bases.

Tabela 6 – Bases de teste da suíte de avaliação

Base	Origem	O que testa
varejo_padrao	Sintética	Caso comum: uma aba, com nomes de colunas descritivos em português
consultoria_abreviada	Sintética	Duas abas (cadastro e desligamentos), com nomes de colunas abreviados
startup_combinada	Sintética	Cargo e nome em uma única coluna e idade informada diretamente
industria_minima	Sintética	Base mínima, com apenas quatro colunas
somente_folha	Sintética	Apenas folha de pagamento, sem cadastro de funcionários
nao_rh_vendas	Sintética	Planilha de vendas, que não deve ser reconhecida como dado de RH (caso negativo)
dividido_por_ano	Sintética	Duas abas de mesma estrutura, referentes a anos diferentes
nexatech_real	Derivada de base de demonstração do projeto	Seis abas, com colunas já no padrão interno
trilha_real	Derivada de base de demonstração do projeto	Três abas, com nomenclatura muito diferente do padrão interno
adversarial_dificil	Construída para ser difícil	Cabeçalhos ambíguos, identificador que não é a matrícula, cargo e área combinados, datas em formatos mistos, salário com símbolo de moeda e uma aba alheia ao RH com nomes de pessoas

Fonte: Elaborada pelos autores (2026).

As bases da Tabela 6 cobriram o caso comum, variações de nomenclatura, dados combinados, uma base mínima, uma base sem cadastro, um caso negativo e um caso construído para ser difícil. Apesar do sufixo _real, as bases nexatech_real e trilha_real foram derivadas de bases de demonstração do projeto, e não de dados de organizações. Para cada base, a suíte chamou a mesma função de detecção de estrutura utilizada na aplicação e comparou a resposta com o gabarito; quando a aba de cadastro foi identificada, a base seguiu pelo pipeline até o planejamento, e cada gráfico proposto foi submetido ao mesmo validador da aplicação. As métricas foram a acurácia na identificação de bases de RH e na classificação do papel das abas, a revocação e a precisão do mapeamento de colunas, calculadas apenas sobre os campos do gabarito, e a taxa de validade dos gráficos. Para colunas com mais de uma correspondência razoável, como “Loja”, que podia representar a área ou a filial, as alternativas aceitas foram registradas no gabarito.

4. TESTES E RESULTADOS

Esta seção apresentou e discutiu os resultados dos testes automatizados, da comparação entre as implementações, das execuções da suíte de avaliação realizadas entre 8 e 30 de setembro de 2026, do tratamento de dados e das medições de desempenho.

4.1 Testes automatizados

Os 67 testes automatizados, 33 da camada de limpeza e 34 do motor de cálculo, foram executados em 29 de outubro de 2026 e todos foram aprovados, em cerca de 0,3 s. Na camada de limpeza, os testes cobriram formatos de número, data e mês, marcadores de ausência, unificação de valores, inclusive diante de falhas e respostas inválidas da IA, relatório de qualidade e intervalos de datas; no motor, cobriram agregações, filtros, cruzamento entre tabelas, validação das especificações, privacidade de salários e do perfil enviado à IA e os cálculos de custo e de simulação. Esses testes verificaram a implementação nos casos previstos, e não a capacidade da IA de interpretar bases desconhecidas, avaliada nas subseções seguintes.

4.2 Consistência entre as implementações em JavaScript e Python

Como verificação independente do motor de cálculo, os valores produzidos pela implementação principal foram comparados aos da implementação em Python para uma mesma base de teste. Se as duas implementações, escritas em linguagens distintas, divergissem,

ao menos uma delas conteria erro. A Tabela 7, a seguir, apresentou exemplos dos valores comparados.

Tabela 7 – Valores calculados de forma independente pelas duas implementações

Indicador calculado	JavaScript	Python
Funcionários ativos	358	358
Funcionários ativos na área de Engenharia	102	102
Custo total de folha, gênero masculino	R$ 25.542.053,10	R$ 25.542.053,10
Custo total de folha, gênero feminino	R$ 20.103.926,97	R$ 20.103.926,97

Fonte: Elaborada pelos autores (2026).

Conforme a Tabela 7, os valores foram idênticos em todos os indicadores comparados, inclusive nos que exigiram o cruzamento entre a folha de pagamento e o cadastro de funcionários. Esse resultado forneceu evidência de que a lógica do motor foi implementada corretamente, e não apenas de que a implementação em JavaScript produziu resultados consistentes consigo mesma. Contudo, a comparação abrangeu apenas um conjunto restrito de indicadores e uma única base.

4.3 Detecção de estrutura das bases

A execução mais recente da suíte foi realizada em 30 de setembro de 2026 com o modelo Claude Sonnet 5, da Anthropic. A Tabela 8, a seguir, apresentou os resultados agregados dessa execução.

Tabela 8 – Resultados agregados da suíte de avaliação na execução de 30 de setembro de 2026

Métrica	Resultado	Base de cálculo
Acurácia na identificação de bases de RH	100%	10 de 10 bases
Acurácia na classificação do papel das abas	100%	19 de 19 abas
Revocação do mapeamento de colunas	100%	145 de 145 campos previstos, em 18 abas
Precisão do mapeamento de colunas	100%	145 de 145 mapeamentos tentados
Taxa de validade dos gráficos	100%	92 de 92 gráficos, em 8 planos

Fonte: Elaborada pelos autores (2026).

Conforme a Tabela 8, o modelo não errou nenhuma das métricas avaliadas. Ele reconheceu a planilha de vendas como alheia ao domínio de RH e, na base difícil, classificou como não pertencente ao domínio a aba de aprovação de orçamento, mesmo contendo nomes de pessoas, além de mapear corretamente a aba de cadastro com cabeçalhos ambíguos. A Tabela 9, a seguir, detalhou os resultados por base.

Tabela 9 – Resultados por base de teste na execução de 30 de setembro de 2026

Base	Abas avaliadas	Papel correto	Campos corretos	Gráficos válidos
varejo_padrao	1	1	8 de 8	10 de 10
consultoria_abreviada	2	2	14 de 14	14 de 14
startup_combinada	1	1	8 de 8	13 de 13
industria_minima	1	1	4 de 4	6 de 6
somente_folha	1	1	6 de 6	–
nao_rh_vendas	–	–	–	–
dividido_por_ano	2	2	14 de 14	9 de 9
nexatech_real	6	6	61 de 61	17 de 17
trilha_real	3	3	23 de 23	15 de 15
adversarial_dificil	2	2	7 de 7	8 de 8
Total	**19**	**19**	**145 de 145**	**92 de 92**

Fonte: Elaborada pelos autores (2026).

Na Tabela 9, a base nexatech_real concentrou 61 dos 145 campos avaliados; as duas abas de anos diferentes da base dividido_por_ano receberam o mesmo papel; e a base trilha_real, com nomenclatura muito diferente do padrão interno, foi mapeada sem erros. O relatório gerado automaticamente nessa execução indicou 94,7% de revocação e de precisão, pois a média por aba incluiu, com valor zero, a aba de aprovação de orçamento, que não tinha campos no gabarito. Recalculados a partir dos dados brutos, apenas sobre as 18 abas com campos previstos, os resultados foram os da Tabela 8, e o cálculo do *script* foi corrigido em seguida.

4.4 Planejamento do dashboard e evolução entre execuções

Nos oito planos avaliados, os 92 gráficos propostos referenciaram campos e tabelas existentes e foram aprovados pelo validador, o que indicou que o perfil estatístico foi suficiente para que o modelo restringisse as propostas ao que a base sustentava. A base composta apenas por folha de pagamento não passou pelo planejamento, pois a suíte exigia a aba de cadastro para essa etapa, ainda que a aplicação tenha passado a tratar o cadastro como opcional, como descrito na Seção 4.5; por isso, houve oito planos para nove bases de RH. O número de gráficos acompanhou a riqueza das bases, de seis na base mínima a 17 na base com seis abas. A taxa de validade mediu se cada gráfico podia ser calculado, e não se ele era o mais útil para o usuário, de modo que a utilidade apontada por Fernandes et al. (2025) para os dashboards permaneceu sem avaliação sistemática.

A suíte foi executada seis vezes ao longo do desenvolvimento, sempre com o mesmo modelo: três vezes com sete bases, duas com nove e uma com dez. Em todas, a identificação das bases, a classificação das abas e o mapeamento de colunas não registraram erros. Na primeira execução, o planejamento falhou em todas as bases por uma referência a uma variável inexistente no código que montava a instrução enviada ao modelo; o erro foi corrigido, o que mostrou que a suíte também funcionou como teste de integração. O número de gráficos variou entre execuções com as mesmas bases, de 53 para 49 com sete bases e de 76 para 79 com nove, o que indicou que as respostas do modelo não eram determinísticas. Ji et al. (2023) associaram a aleatoriedade das estratégias de amostragem usadas na geração de texto a um maior risco de alucinação, o que reforçou a necessidade do validador, embora todos os gráficos tenham sido válidos nessas execuções.

4.5 Tratamento de dados e desempenho

A base de demonstração com 359 erros injetados foi processada pela plataforma, e o relatório de qualidade passou a mostrar ao usuário quais valores foram convertidos, unificados ou descartados. Durante esse processamento, uma lacuna em um dos meses do gráfico de custo de folha revelou que um formato de competência mensal não era reconhecido. A normalização de competências foi então implementada, a série passou a ser exibida de forma contínua, e o caso foi incorporado aos testes automatizados. Contudo, a eficácia da limpeza em relação ao gabarito, isto é, a proporção dos 359 erros efetivamente detectados ou corrigidos, não foi quantificada de forma sistemática, o que constituiu uma limitação deste trabalho.

Durante o desenvolvimento, bases deliberadamente incompletas – apenas com recrutamento, apenas com folha ou com recrutamento e desligamentos – revelaram que partes do sistema assumiam a existência obrigatória do cadastro de funcionários. A arquitetura foi revisada para tratar o cadastro como opcional, e os três cenários passaram a gerar dashboards com os indicadores que cada base sustentava. Quanto ao desempenho, com uma base de folha replicada, a limpeza levou cerca de 52 ms para 6,8 mil linhas, 346 ms para 68 mil e 1,6 s para 342 mil, enquanto o cálculo de um gráfico variou entre 1 ms e 126 ms, sendo mais lento quando exigia o cruzamento com o cadastro. Esses tempos foram indicativos, pois dependiam do equipamento, que não foi registrado de forma padronizada.

4.6 Discussão geral e limitações

Os resultados indicaram que a arquitetura proposta funcionou de forma integrada: a IA interpretou corretamente a estrutura das dez bases de teste, o validador não precisou descartar

nenhum gráfico, e o código calculou os valores de forma reprodutível, o que foi corroborado, nos indicadores comparados, pela concordância entre as implementações em JavaScript e Python. A separação entre decisão e cálculo permitiu aproveitar a capacidade de interpretação do modelo sem atribuir a ele a produção dos números, o que respondeu ao risco de alucinação discutido na Seção 2.3 (JI et al., 2023) e limitou a propagação de erros da IA para os indicadores, risco que Qu et al. (2026) associaram ao comprometimento de dados em sistemas de interação entre pessoas e IA.

Em relação às abordagens da Tabela 1, a plataforma ocupou uma posição intermediária. Assim como as ferramentas de geração de visualizações por LLM (DIBIA, 2023), dispensou a preparação prévia dos dados por especialistas, exigida pelos assistentes de BI de mercado (MICROSOFT, [2026]; TABLEAU, [2026]); assim como as estratégias baseadas em saídas estruturadas e regras estritas (RUMIANTSAU et al., 2024), restringiu o modelo a especificações verificáveis. O preço dessa escolha foi a menor expressividade: pedidos que não cabiam no vocabulário do motor foram rejeitados, em vez de atendidos por código gerado livremente.

Quanto aos objetivos específicos, a detecção de estrutura (i), a geração de dashboards calculados por código (iii) e a avaliação da precisão da IA (v) foram verificadas quantitativamente pela suíte e pelos testes. A limpeza (ii) e a proteção de dados (iv) foram implementadas e testadas, mas ainda dependiam de etapas complementares: no caso da limpeza, a medição de sua eficácia contra o gabarito de 359 erros e a avaliação da unificação por significado com a API real; no caso da proteção de dados, a remoção de nomes da amostra enviada na detecção de estrutura.

Os resultados também evidenciaram outras limitações. O corpus de avaliação foi pequeno, construído pelos próprios autores e sem bases reais de organizações; o gabarito foi elaborado por um único avaliador; e apenas um modelo foi avaliado. As projeções, o simulador e a estimativa de risco não foram avaliados quanto à exatidão, e a utilidade dos dashboards para gestores e o tempo economizado em relação à preparação manual também não foram medidos. O esquema de seis tabelas limitou a plataforma ao domínio de RH, e a chave de acesso exposta no navegador restringiu o uso a ambientes locais. Por fim, como observaram Arnaboldi, Robbiani e Carlucci (2021), ferramentas de autoatendimento não eliminavam a necessidade de conhecimento técnico, e a revisão do mapeamento pelo usuário continuou sendo um ponto de controle necessário.

5. CONSIDERAÇÕES FINAIS

Este trabalho desenvolveu e avaliou uma plataforma de *People Analytics* que automatizou o pipeline de dados para usuários de negócio, aplicada a bases de Recursos Humanos. A plataforma, executada no navegador, integrou a leitura de planilhas, a detecção da estrutura dos dados por um modelo de linguagem, a limpeza e a padronização determinísticas, o planejamento do dashboard pela IA, a validação das especificações, o cálculo de todos os valores por código e recursos estatísticos de projeção e simulação.

Os resultados obtidos atenderam aos objetivos estabelecidos, com as ressalvas apontadas na Seção 4.6 para a limpeza e a proteção de dados. Na execução mais recente da suíte, o modelo identificou corretamente o domínio das dez bases de teste, o papel das 19 abas e os 145 mapeamentos de colunas previstos, e os 92 gráficos propostos foram aprovados pelo validador; nas seis execuções, não houve erro de classificação e, sempre que o planejamento foi concluído, todos os gráficos foram válidos. Os 67 testes automatizados foram aprovados, as implementações em JavaScript e Python produziram valores idênticos nos indicadores comparados, e, em medição indicativa, a limpeza de uma base com 342 mil linhas levou cerca de 1,6 s.

Como contribuição, o trabalho documentou uma arquitetura em que a IA decidiu o que exibir e o código calculou os valores exibidos, com validação determinística de toda saída do modelo e uma suíte de avaliação com procedimento reprodutível. Na visão dos autores, essa abordagem mostrou-se uma forma viável de conciliar a flexibilidade dos modelos de linguagem com a confiabilidade exigida por ferramentas de apoio à decisão e de aproximar o *People Analytics* de usuários de negócio. Ainda assim, o corpus reduzido, construído pelos próprios autores, e a ausência de testes com usuários limitaram a generalização dos resultados.

6. PROPOSTAS DE TRABALHOS FUTUROS

As propostas a seguir decorreram das limitações discutidas na Seção 4.6. Quanto à avaliação, sugerem-se: quantificar a eficácia da limpeza com o gabarito de 359 erros; avaliar a unificação por significado com a API real; ampliar o corpus com bases reais anonimizadas e com gabarito revisado por mais de um avaliador; repetir a avaliação com outros modelos e com várias execuções por configuração; e avaliar a exatidão das projeções e da estimativa de risco.

Quanto à plataforma e ao seu uso, sugerem-se: remover os nomes da amostra enviada na detecção de estrutura; adotar um servidor intermediário para proteger a chave de acesso; e realizar um estudo de usabilidade com profissionais de RH, submetido previamente a um

Comitê de Ética em Pesquisa, para medir a utilidade dos dashboards e o tempo economizado em relação à preparação manual.

REFERÊNCIAS

ANGRAVE, D.; CHARLWOOD, A.; KIRKPATRICK, I.; LAWRENCE, M.; STUART, M. HR and analytics: why HR is set to fail the big data challenge. **Human Resource Management Journal**, v. 26, n. 1, p. 1–11, 2016. DOI: 10.1111/1748-8583.12090.

ARNABOLDI, M.; ROBBIANI, A.; CARLUCCI, P. On the relevance of self-service business intelligence to university management. **Journal of Accounting & Organizational Change**, v. 17, n. 1, p. 5–22, 2021. DOI: 10.1108/JAOC-09-2020-0131.

BAHAMÓN-MONJE, A. F.; COLLAZOS-ESCOBAR, G. A.; GUTIÉRREZ-GUZMÁN, N. Data-driven modeling of water adsorption isotherms in cocoa beans: dataset and Python-based machine learning tools for multivariate analysis and storage management. **Data in Brief**, v. 65, artigo 112616, 2026. DOI: 10.1016/j.dib.2026.112616.

BALA, M.; BOUSSAID, O.; ALIMAZIGHI, Z. A fine-grained distribution approach for ETL processes in big data environments. **Data & Knowledge Engineering**, v. 111, p. 114–136, 2017. DOI: 10.1016/j.datak.2017.08.003.

BARENDS, E.; ROUSSEAU, D. M. **Evidence-based management**: how to use evidence to make better organizational decisions. London: Kogan Page, 2018.

BOUAZIZ, S.; NABLI, A.; GARGOURI, F. Extending ETL for NoSQL: a validated framework for document-oriented warehousing. **Procedia Computer Science**, v. 270, p. 4114–4123, 2025. DOI: 10.1016/j.procs.2025.09.536.

BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD). Brasília, DF: Presidência da República, 2018. Disponível em: https://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/l13709.htm. Acesso em: 29 out. 2026.

COLOMBO, A.; BERNASCONI, A.; CERI, S. An LLM-assisted ETL pipeline to build a high-quality knowledge graph of the Italian legislation. **Information Processing & Management**, v. 62, n. 4, artigo 104082, 2025. DOI: 10.1016/j.ipm.2025.104082.

DEB NATH, R. P.; HOSE, K.; PEDERSEN, T. B.; ROMERO, O. SETL: a programmable semantic extract-transform-load framework for semantic data warehouses. **Information Systems**, v. 68, p. 17–43, 2017. DOI: 10.1016/j.is.2017.01.005.

DIBIA, V. LIDA: a tool for automatic generation of grammar-agnostic visualizations and infographics using large language models. In: ANNUAL MEETING OF THE ASSOCIATION FOR COMPUTATIONAL LINGUISTICS, 61., 2023, Toronto. **Proceedings** [...]. Volume 3: System Demonstrations. Toronto: Association for Computational Linguistics, 2023. p. 113–126. DOI: 10.18653/v1/2023.acl-demo.11.

FERNANDES, I. et al. A data pipeline for support Decision Support System in Neonatology. **Procedia Computer Science**, v. 272, p. 570–575, 2025. DOI: 10.1016/j.procs.2025.10.249.

GARTNER. Citizen developer. In: GARTNER. **Gartner Glossary**. Stamford: Gartner, [2026]. Disponível em: https://www.gartner.com/en/information-technology/glossary/citizen-developer. Acesso em: 29 out. 2026.

IMHOFF, C.; WHITE, C. **Self-service business intelligence**: empowering users to generate insights. [S. l.]: TDWI, 2011. (TDWI Best Practices Report, third quarter 2011). Disponível em: https://tdwi.org/research/2011/07/best-practices-report-q3-self-service-business-intelligence.aspx. Acesso em: 29 out. 2026.

JI, Z. et al. Survey of hallucination in natural language generation. **ACM Computing Surveys**, v. 55, n. 12, artigo 248, 2023. DOI: 10.1145/3571730.

LUO, Y.; QIN, X.; TANG, N.; LI, G. DeepEye: towards automatic data visualization. In: IEEE INTERNATIONAL CONFERENCE ON DATA ENGINEERING, 34., 2018, Paris. **Proceedings** [...]. [S. l.]: IEEE, 2018. p. 101–112.

LUO, Y.; TANG, N.; LI, G.; CHAI, C.; LI, W.; QIN, X. Synthesizing natural language to visualization (NL2VIS) benchmarks from NL2SQL benchmarks. In: INTERNATIONAL CONFERENCE ON MANAGEMENT OF DATA (SIGMOD), 2021. **Proceedings** [...]. New York: ACM, 2021.

MARLER, J. H.; BOUDREAU, J. W. An evidence-based review of HR Analytics. **The International Journal of Human Resource Management**, v. 28, n. 1, p. 3–26, 2017.

MICROSOFT. Copilot in Power BI tutorial: prepare semantic model for AI. In: MICROSOFT. **Microsoft Learn**. Redmond: Microsoft, [2026]. Disponível em: https://learn.microsoft.com/en-us/power-bi/create-reports/tutorial-copilot-power-bi-prepare-model. Acesso em: 29 out. 2026.

MIGUEL, P. A. C. (org.). **Metodologia de pesquisa em engenharia de produção e gestão de operações**. 2. ed. Rio de Janeiro: Elsevier: ABEPRO, 2012.

MOZILLA. IndexedDB API. In: MOZILLA. **MDN Web Docs**. [S. l.]: Mozilla, [2026]. Disponível em: https://developer.mozilla.org/en-US/docs/Web/API/IndexedDB_API. Acesso em: 29 out. 2026.

OPENJS FOUNDATION. Test runner. In: OPENJS FOUNDATION. **Node.js documentation**. [S. l.]: OpenJS Foundation, [2026]. Disponível em: https://nodejs.org/api/test.html. Acesso em: 29 out. 2026.

PROJECT MANAGEMENT INSTITUTE (PMI). **PMI Citizen Developer**. Newtown Square: PMI, [2026]. Disponível em: https://www.pmi.org/citizen-developer. Acesso em: 29 out. 2026.

QU, M.; LI, S.; JIANG, T.; FANG, Z.; GUO, P.; LI, Q. Preventing data-driven risk propagation in human–artificial intelligence interaction: a scenario security architecture. **Computers in Industry**, v. 177, artigo 104458, 2026. DOI: https://doi.org/10.1016/j.compind.2026.104458.

REACT: the library for web and native user interfaces. [S. l.], [2026]. Disponível em: https://react.dev/. Acesso em: 29 out. 2026.

RECHARTS: a composable charting library built on React components. [S. l.], [2026]. Disponível em: https://github.com/recharts/recharts. Acesso em: 29 out. 2026.

RUMIANTSAU, M.; VERTSEL, A.; HRYTSUK, I.; BALLAH, I. Beyond fine-tuning: effective strategies for mitigating hallucinations in large language models for data analytics. **arXiv**, 2410.20024, 2024. Disponível em: https://arxiv.org/abs/2410.20024. Acesso em: 29 out. 2026.

SANYADANAM, A.; SRIRAMA, S. N. Serverless data pipeline architecture supporting distributed machine learning on fog devices. **Computer Communications**, v. 252, artigo 108505, 2026. DOI: 10.1016/j.comcom.2026.108505.

SATYANARAYAN, A.; MORITZ, D.; WONGSUPHASAWAT, K.; HEER, J. Vega-Lite: a grammar of interactive graphics. **IEEE Transactions on Visualization and Computer Graphics**, v. 23, n. 1, p. 341–350, 2017. DOI: 10.1109/TVCG.2016.2599030.

SHEETJS. **SheetJS Community Edition**: documentation. [S. l.]: SheetJS LLC, [2026]. Disponível em: https://docs.sheetjs.com/. Acesso em: 29 out. 2026.

TABLEAU. Create metrics with Tableau Pulse. In: TABLEAU. **Tableau Help**. [S. l.]: Salesforce, [2026]. Disponível em: https://help.tableau.com/current/online/en-us/pulse_create_metrics.htm. Acesso em: 29 out. 2026.

WANG, R. Y.; STRONG, D. M. Beyond accuracy: what data quality means to data consumers. **Journal of Management Information Systems**, v. 12, n. 4, p. 5–33, 1996.